Path Planning:

Problem Definition:

The path planning subsystem is required to formulate a feasible trajectory that considers time and navigates around static obstacles, leading the differential-drive robot from its starting point through a series of planned plant inspection waypoints. Along with a motion-mode flag indicating whether the robot is rotating in place or translating along a straight-line segment, the output trajectory is a discretised sequence of states (x,y,θ) sampled at a fixed time interval $\Delta t=0.1s$.

A key constraint driving this problem definition is the capability of the low-level motion controller. Differential-drive robots are most naturally controlled through decoupled linear and angular velocity commands [1], and in this implementation the controller was designed to reliably track only two primitive motions: pure in-place rotation and straight-line translation at constant heading. Therefore, the path planner must generate paths that are exclusively composed of straight-line segments connecting waypoints, with heading changes occurring solely at the beginning or end of each segment.

According to the project's baseline goals, the planner must also meet the operational needs of the monitoring task, which include visiting each of the six plant, taking pictures at each plant, and completing the tour without human involvement.

Environment Modelling:

Figure 2.3.1 illustrates the TS Explore room (10.5 m x 14 m), which is modelled in the world coordinate frame established by the ArUco marker arrangement provided for the final demonstration. In accordance with the known obstacle geometry, the workspace is divided into free and occupied cells using a standard representation for indoor mobile robot navigation that discretises the workspace into a 2D occupancy grid to represent the room. The tables are depicted in the grid as rectangular areas to signify the robot's footprint and the ±5 cm tolerance in placement, aligning with the stationary obstacles in the room. The six plant sites (P1–P6) are fixed goal coordinates within the layout specification's world frame, derived directly from the measured plant positions given in the layout specification .

For the motion controller to execute effectively, this marker-referenced, grid-based representation provides a unified coordinate frame that facilitates consistent communication of the planned waypoints, obstacle thresholds, and destination points, utilized by both the global path planner and the trajectory generation process.

Design Decision Justification:

The trajectory generation method was refined in three stages, each of which was sparked by restrictions that became exposed when the differential-drive controller performed the planned route. Each iteration is discussed in this section, along with a link between the experimental behavior seen and the underlying planning theory, and the rationale behind the ultimate design.

- *Iteration 1: A* with CHOMP-style smoothing:* The first implementation utilized the A* search method [1], which creates an optimal-cost route when a valid heuristic is applied, yet restricts movement to eight grid-connected directions and results in pathways with sudden, axis-aligned directional changes. To make this into a trajectory that a continuously moving robot could follow, a CHOMP-style gradient-based smoothing stage (Covariant Hamiltonian Optimization for Motion Planning), was used [2]. By descending a cost functional that blends a smoothness term (penalizing curvature) with an obstacle-cost term (penalizing proximity to occupied cells), CHOMP repeatedly perturbs the path, resulting in a locally smooth, collision-free path.
  Figure 1 shows the x(t), y(t), and φ(t) profiles as well as the room map on which the resultant trajectory is overlaid. The heading profile φ(t) shows numerous near-instantaneous transitions of around ±180° even if the position profiles x(t) and y(t) are visually smooth. These discontinuities appear when the CHOMP-smoothed road turns back on itself close to obstacle boundaries (for example, while navigating through rows of tables), a setup that the smoothing cost cannot completely fix without breaking the obstacle-avoidance condition. The jerk-limited heading controller cannot follow φ_ref inside the available bandwidth, which results in the significant heading excursions seen in Figure 1. This is because φ_ref(t) is determined from the path's local tangent

direction through finite difference, and any near-reversal in direction generates a step discontinuity in φ_ref.
When tested on the actual robot, instead of the smooth and gradual movement envisioned by the CHOMP optimization, there were sudden, aggressive turning actions occurring close to the plant sites. The low-level controller lacked the ability to track the smoothed path's continuously curving heading reference since it was only built to track straight-line and in-place rotation primitives. Instead, it attempted to fix the large heading error directly, causing swift, jerky rotations at the exact places where cautious, slow manoeuvring around the plants was necessary. This real-world failure mode provided the practical motivation for abandoning curvature-based path smoothing entirely, and is discussed further in Iterations 2 and 3.

- ***Iteration 2: Lazy Theta****:* The planner was replaced by Lazy Theta*, an any-angle route planning variant of Theta* [3], [4], in order to alleviate the zig-zag artefacts that are intrinsic to grid-constrained A* paths. Theta*, and its lazy varient, unlike A*, allow a node's parent to be any vertex with line-of-sight visibility, not only its grid neighbors. As a result, the resulting route can be comprised of lengthy, direct straight-line sections rather than staircase-like sequences of unit steps. The "lazy" evaluation defers line-of-sight checks until a path is selected for expansion, substantially reducing computational cost [4].
The resulting trajectory, as well as the accompanying x(t), y(t), and φ(t) profiles, are shown in Figure 2. Although the general course is more straight than the A*+CHOMP output, the heading profile (t) now shows persistent high-frequency oscillation (chattering) across multiple segments. This conduct is a necessary consequence of defining the route as a polyline: the reference heading (φ ref) is discontinuous at each vertex where the direction of the path changes. In an effort to follow this discontinuous reference, the heading controller repeatedly overshoots and corrects around each transition, resulting in the ripple pattern seen in φ(t), using a continuous (jerk-limited) control legislation. Since the controller used in this project was not designed to track a continuously varying heading reference — only discrete in-place rotations and constant-heading translations — this chattering could not be eliminated by re-tuning the controller gains alone.

- Iteration 3: Straight-line interpolation with quintic velocity profile: The outcomes of Iterations 1 and 2 demonstrate that, independent of the planning technique employed, each polyline path inevitably generates discontinuous headings at each vertex. In order to accommodate this, the final design breaks down motion at each waypoint into two phases: a straight-line translation phase at constant heading with a quintic (minimum-jerk) velocity profile, and a turn-in-place phase that rotates at constant angular velocity to face the next segment. On the actual robot, this eliminated both Iteration 1's overshoots and Iteration 2's chattering because every commanded primitive corresponds to one that the controller is programmed to track — trading path optimality for execution reliability. This method was chosen as the final trajectory generating methodology, which is detailed in Section 2.4.4, because to its successful performance in the first demonstration and the time constraints for the remaining portion of the project.

Global Path Planning and Trajectory Design

The final trajectory generation algorithm converts an ordered list of waypoints into a discretised sequence of robot states (x, y, θ), sampled at a fixed interval $\Delta t = 0.1$ s, along with a motion-mode flag. This process builds upon the design rationale provided in the Design Decision Justification section. As a result, the trajectory generator interprets the global path as a series of straight-line segments that link successive waypoints. Each segment is processed one at a time, resulting in two different phases of motion: a turn-in-place phase and a straight-line translation phase.

Initialisation:

In order to give an initial settling period, the trajectory starts with the robot motionless in its initial posture $(x_0, y_0, \theta_0) = (0.2, 0.2, \pi/2)$ for one second. The motion-mode flag is 1 throughout this period.

Turn-in-place stage

The direction of the displacement vector is the necessary heading for the segment between waypoint i and waypoint i+1:

θ_target = arg(Δx + jΔy), where $\Delta x = x_{i+1} - x_i$ and $\Delta y = y_{i+1} - y_i$ (i.e., the angle of the vector (Δx, Δy) measured from the positive x-axis).

The signed angular distance between the current heading θ_cur and θ_target, measured on the interval (−π, π), is the necessary heading change.

Δθ = θ_target − θ_cur, with 2π added or subtracted to have Δθ ∈ (−π, π).

The time needed for this rotation, assuming a constant angular speed ω = v_turn, is:

T_turn = |Δθ| / ω

Since the trajectory is sampled at discrete intervals of Δt, the duration is rounded up to the next whole number of time steps.

N_turn is equal to the smallest integer > T_turn/Δt.

At every step k = 1, 2,..., N_turn, the heading moves in the direction of Δθ at a rate of ω:

Once more normalized to (−π, π), θ_k = θ_cur + sign(Δθ) · ω · Δt · k

The motion-mode flag is 0 and the robot's position stays fixed at (x_i, y_i) during this phase. The final step k = N_turn may slightly exceed θ_target since N_turn is an integer number of whole steps and T_turn is typically not an exact multiple of Δt. By explicitly setting the heading to θ_target before to the start of the straight-line phase, this residual inaccuracy is eliminated.

Straight-line Phase

The robot now faces θ_target and moves straight from (x_i, y_i) to (x_{i+1}, y_{i+1}), a distance:

Along the unit direction vector (cos θ_target, sin θ_target), d = √(Δx² + Δy²).

The robot's speed follows a smooth acceleration–cruise–deceleration profile instead of starting at a steady speed, allowing velocity (and thus acceleration) to fluctuate continually without sudden jumps. The quintic (fifth-order) polynomial is utilized to accomplish this:

$10\tau^3$ = s(τ) For τ ∈ [0, 1], $-15\tau^1 + 6\tau\delta$

This function rises gradually from s(0) = 0 to s(1) = 1, with both its first derivative (rate of change) equal to zero at τ = 0 and τ = 1. This means that there are no abrupt jerks at the beginning or conclusion of each ramp.

Since the motion is separated into three stages, T_accel (acceleration), T_cruise (constant speed), and T_accel again (deceleration), the speed at time t since the segment's beginning is:

$v(t) = v_{linear} \cdot s(t / T_{accel}) \qquad \text{for } 0 \le t \le T_{accel}$

For $T_{accel} < t \le T_{accel} + T_{cruise}$, $v(t) = v_{linear}$

For $T_{accel} + T_{cruise} < t \le T_{total}$, $v(t) = v_{linear} \cdot [1 - s((t - T_{accel} - T_{cruise}) / T_{accel})]$

The area beneath v(t) from 0 to T_accel represents the distance traveled during the acceleration phase, which can be used to calculate T_accel.

distance travelled = $v_{linear} \cdot T_{accel} \cdot$ (area under s over [0,1]) = $v_{linear} \cdot T_{accel} \cdot (1/2)$

If the selected acceleration distance d_accel must be equal to this distance, then:

$T_{accel} = 2 \cdot d_{accel} / v_{linear}$

If the segment is shorter than $2 \cdot d_{accel}$, the acceleration distance is lowered to half the segment length. This prevents the robot from attempting to accelerate across a longer distance than the segment includes.

The smaller of d_accel and d/2 is $d_{accel,eff}$.

After taking acceleration and deceleration into account, the remaining portion of the segment is the cruising distance:

$d_{cruise} = d - 2 \cdot d_{accel,eff}$, $T_{cruise} = d_{cruise} / v_{linear}$, and the segment's overall duration is:

$T_{total} = 2 \cdot T_{accel} + T_{cruise}$

N_{move} = smallest integer >= $T_{total}/\Delta t$ discrete time steps are used to sample this segment. The distance traveled at step k (for $t = k \cdot \Delta t$, capped at T_{total}) is given by the speed v(t) above, $\Delta s = v(t) \cdot \Delta t$.

The total distance traveled (s) is the running sum of these increments, capped at d. At each stage, the robot's position is then:

$(x, y) = (x_i, y_i) + s \cdot (\cos \theta\_target, \sin \theta\_target)$ with the motion-mode flag set to 1 and the heading fixed at θ_target. A last point at the correct coordinates $(x\_\{i+1\}, y\_\{i+1\})$ is attached after the loop reaches the conclusion of the section, ensuring the route goes through each waypoint exactly.

The following parameters were used to construct the trajectory: d_accel = 0.5 m, Δt = 0.1 s, v_linear = 0.5 m/s, and v_turn = 1.0 rad/s and have been chosen by trial and error.

Output format:

The trajectory is expressed as a series of rows with five values in each row:

x position (metres)

y position (metres)

θ heading (radians)

Motion-mode flag: 0 indicates stationary rotation, 1 indicates translation in a straight line

Plant-marker: n = Plant n, 0 = no targeted plant

Experimental results:

This part summarizes the findings of both simulation and physical robot testing for the final trajectory generation approach described in the Global Path Planning and Trajectory Generation section. 1

Simulated Trajectory Profiles

1521 discretised points at a 0.1 s sampling interval make up the generated trajectory. Figure 10 illustrates the resulting x(t), y(t), and θ(t) profiles. The quintic acceleration–cruise–deceleration speed profile applied along each straight-line segment is represented by the position profiles x(t) and y(t), which exhibit smooth, S-shaped transitions between successive plateaus. Each transition starts and ends gently (zero acceleration at the endpoints) and reaches a constant rate of change during the cruise phase.

2.6.5.2 The heading profile θ(t) is piecewise constant, and only the turn-in-place phases (motion-mode flag = 0) exhibit instantaneous changes. The main qualitative distinction between this version and the two previous iterations

covered in the Design Decision Justification section is that each heading change is a discrete, controller-tracked rotation as opposed to the massive heading overshoots seen with A*+CHOMP smoothing or the persistent high-frequency chattering seen with Lazy Theta*. There is no unintentional heading drift during straight-line translation since $\theta(t)$ stays perfectly constant between these transitions.

2.6.5.2 Trajectory Overlaid on the Room Map:

The proposed route is superimposed on the TS Explore room architecture in Figure [y]. The path starts at the start position near the bottom-left corner, goes to Plant 1 in the top-left corner, then travels along the upper row of tables in a corridor to the area of Plants 2, 3, and 4. It then crosses to the right side of the room to visit Plants 6 and 5 in the upper-right and lower-right regions, respectively, before returning to the start position. Throughout, the path stays in the open space between the table rows; no part of it crosses an obstacle area.

2.6.5.3 Physical Robot Testing:

The trajectory generation approach was implemented on the actual robot platform and tested during the initial demonstration. The turn-in-place plus straight-line approach was reliably implemented by the low-level controller, in contrast to the A*+CHOMP and Lazy Theta* approaches discussed in the Design Decision Justification section, which both resulted in unreliable heading tracking on hardware, exhibiting either aggressive turning near the plants or persistent oscillatory corrections at path vertices. At every waypoint that required a heading change, the robot was shown to smoothly rotate to face the following segment, and then go down the straight-line section at the specified speed profile without chattering or overshooting.

The final trajectory generation method produces a time-parameterized sequence of states at a fixed 0.1 s interval, limits heading changes to discrete in-place rotations, and successfully navigates the robot between all necessary plant-inspection waypoints while avoiding the known static obstacles, all of which the results show meet the requirements outlined in the Problem Definition. The choice to keep this method as the final trajectory generation strategy was supported by the first demonstration's strong performance and the project's short timeline.

Limitations and Improvements:

While the straight-line interpolation with quintic velocity profile technique performed well in practice, some drawbacks were discovered during the project.

The biggest drawback is that the trajectory is completely static and pre-planned; all trajectory points and the waypoint sequence are calculated offline before the robot starts moving, and it has no ability to adjust to changes in the environment while it is in motion. The robot has no way of detecting or avoiding an unexpected barrier, such a chair or a person, along a scheduled straight-line stretch, thus it will proceed along the pre-planned path.

A Second constraint is the waypoint sequence, which was determined manually by inspecting the room layout and predicting safe pathways between obstacles. This method is time-consuming and prone to mistakes. Additionally, it cannot be transferred to a different setting without starting over from scratch.

Lastly, the robot's physical dimensions are not taken into consideration by the trajectory. There is no defined safety margin to ensure that the robot's body clears the tables on either side of a hallway; instead, the intended path runs through the geometric center of the free space between obstructions.

Improvements:

The addition of a real-time obstacle avoidance layer based on the onboard LiDAR sensor would be the most significant enhancement. The LiDAR point cloud could be examined for obstructions within a look-ahead distance along the current scheduled section at each control cycle. When an obstacle is identified, a local avoidance technique like the Vector Field Histogram (VFH) [4] or the Dynamic Window Approach (DWA) [3] could calculate a safe velocity instruction that guides the robot around the impediment while staying as near to the global planned course as feasible.

Future work could revisit the curved-path planning approaches investigated in Iterations 1 of the design process to increase time efficiency, assuming that the low-level controller is modified or enhanced to accommodate curvature tracking. A differential-drive robot would benefit greatly from a Dubins route formulation [2], which generates minimum-length paths made up of straight lines and circular arcs that adhere to a maximum curvature restriction, thus removing the need for stop-and-turn maneuvers.

The manual waypoint defining phase would be eliminated and the system would be much more transferable to a new room layout if waypoints were generated automatically using a global path planner like A*, applied directly to the occupancy grid and updated plant positions. Once the controller limitation is fixed, this feature, which was only partially investigated in the previous design rounds, may be reintroduced.

Finally, integrating a robot footprint inflation layer into the occupancy grid would provide formal assurance that all planned segments have a minimum safe clearance from all obstacles. This would take the place of the existing implicit assumption that a corridor's geometric center may always be safely traversed. It is especially crucial for small spaces between table edges.